Learning, Adaptation, and Generalization for Autonomous Agents and Robots in Open-World Environments

A dissertation submitted by

Shivam Goel

in partial fulfillment of the requirements
for the degree of

Doctor of Philosophy

in

Computer Science

TUFTS UNIVERSITY

May 2026

©Copyright 2025 by Shivam Goel

Advisors: Jivko Sinapov & Matthias Scheutz

Abstract

Autonomous agents and robots deployed in real-world environments must operate under open-world conditions in which unforeseen changes, novel objects, and unexpected task variations are inevitable. However, most contemporary artificial intelligence and robotics systems are developed under closed-world assumptions where environments, tasks, and dynamics encountered during deployment closely match those observed during training. These assumptions limit the ability of intelligent systems to adapt when prior knowledge becomes insufficient and to generalize robustly across new situations. This dissertation develops methods for enabling autonomous agents and robots to operate effectively in open-world environments through learning, adaptation, and generalization. To this end, it introduces benchmark environments that systematically inject novelty and enable rigorous evaluation of novelty-aware agents. It further develops adaptive planning-learning systems that integrate task and motion planning with reinforcement learning-based recovery mechanisms, allowing agents to detect execution failures and adapt their behavior when previously learned knowledge becomes insufficient. Finally, it demonstrates how robots can acquire generalizable interaction capabilities through object-centric and force-based reinforcement learning policies that enable sustained-contact manipulation across diverse objects and tasks. Together, these contributions provide a unified perspective on building autonomous agents and robots capable of operating in open-world environments. By combining benchmark-driven evaluation, adaptive planning-learning systems, and generalizable embodied interaction policies, this work advances the development of intelligent systems that can robustly operate in complex, dynamic, and evolving real-world settings.

Acknowledgments

I would like to thank...

Contents

Abstract **ii**

Acknowledgments **iii**

1 Background and Preliminaries **2**

1.1 Symbolic Planning 2

1.2 Reinforcement Learning 3

1.3 Hybrid Planning and Learning 5

List of Figures

Chapter 1

Background and Preliminaries

This chapter introduces the formal frameworks that underpin the methods developed throughout this dissertation. The goal is not to provide a comprehensive survey of the fields, but rather to establish the mathematical models and notation required to describe the proposed algorithms and systems.

The dissertation builds on three core paradigms for sequential decision making: symbolic planning, reinforcement learning, and hybrid planning and learning. These paradigms provide complementary capabilities for reasoning, learning, and adaptation in complex environments.

Section 1.1 introduces the formalism of symbolic planning, including representations of states, predicates, and action operators. Section 1.2 presents reinforcement learning using the Markov Decision Process framework. Section 1.3 introduces hybrid planning and learning architectures that integrate symbolic reasoning with learning-based decision making, and formalizes the notion of open-world environments and novelty.

These formalisms provide the foundation for the methods presented in subsequent chapters. Following this formal background, later sections of this chapter situate the thesis within the broader literature on open-world learning, adaptive agents, and robotics.

1.1 Symbolic Planning

Symbolic planning models sequential decision making using an explicit representation of states, actions, and goals. The environment is described using a first-order language consisting of

predicate symbols and function symbols. Predicate symbols define Boolean properties of objects, while function symbols define numeric quantities.

A fluent is a state variable whose value may change over time. Boolean fluents are obtained by grounding predicate symbols with objects in the domain, and numeric fluents are obtained from function symbols. Let $\mathcal{F}$ denote the set of all fluents. A symbolic state $\tilde{s} \in \tilde{\mathcal{S}}$ is defined as a over $\mathcal{F}$, assigning each Boolean fluent a value in $\{0, 1\}$ and each numeric fluent a value in $\mathbb{R}$. In propositional domains, a symbolic state can be equivalently represented as the set of grounded predicates that are true, with all others assumed false under the closed-world assumption.

The dynamics of the symbolic model are defined by a finite set of operators $\mathcal{O}$. Each operator $o \in \mathcal{O}$ is defined as $o = \langle \psi_o, \omega_o \rangle$, where ψ_o is a logical condition over fluents specifying when the operator is applicable, and ω_o specifies how the state changes after execution. An operator is applicable in a state $\tilde{s}$ if $\tilde{s} \models \psi_o$. Effects modify Boolean fluents through add and delete operations and update numeric fluents through assignments or arithmetic expressions. Operators are typically defined as parameterized schemas and instantiated over objects to produce grounded operators.

A planning task is defined as $\mathcal{T} = \langle \mathcal{F}, \mathcal{O}, \tilde{s}_0, g \rangle$, where $\tilde{s}_0 \in \tilde{\mathcal{S}}$ is the initial state and g is a logical condition over fluents. A plan is a finite sequence of operators $\rho = [o_1, \ldots, o_n]$. Executing a plan from the initial state produces a sequence of symbolic states obtained by sequentially applying applicable operators. A plan is valid if execution from $\tilde{s}_0$ results in a state that satisfies g. Classical symbolic planning assumes deterministic transitions, where the effects of an operator uniquely determine the resulting state.

Planning domains and problems are commonly specified using the Planning Domain Definition Language (PDDL), which provides a standard representation for predicates, functions, operators, and planning tasks [1].

1.2 Reinforcement Learning

Reinforcement learning studies how an agent can learn to make sequential decisions through interaction with an environment in order to maximize cumulative reward. At each time step, the agent observes the current state, selects an action, and receives feedback in the form of a

reward signal.

Reinforcement learning problems are commonly modeled as a Markov Decision Process (MDP), defined as $M = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, $P(s' \mid s, a)$ defines the probability of transitioning to state s' after taking action a in state s, $R(s, a)$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor.

At time step t, the agent is in state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$, receives reward $r_{t+1} = R(s_t, a_t)$, and transitions to a new state s_{t+1} according to $P(\cdot \mid s_t, a_t)$. The Markov property assumes that the transition and reward depend only on the current state and action.

A policy $\pi(a \mid s)$ defines the behavior of the agent by specifying a probability distribution over actions given a state. The objective of reinforcement learning is to find a policy that maximizes the expected cumulative reward.

The return from time step t is defined as $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$. The state-value function is defined as $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$, and the action-value function is defined as $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$.

These value functions satisfy recursive relationships known as the Bellman expectation equations. The state-value function satisfies

$$V^\pi(s) = \mathbb{E}_\pi \left[r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s \right],$$

and the action-value function satisfies

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[r_{t+1} + \gamma \mathbb{E}_{a' \sim \pi(\cdot \mid s_{t+1})} Q^\pi(s_{t+1}, a') \mid s_t = s, a_t = a \right].$$

In discrete settings, the state and action spaces are finite, i.e., $|\mathcal{S}| < \infty$ and $|\mathcal{A}| < \infty$. In this case, value functions such as $V^\pi(s)$ and $Q^\pi(s, a)$ can be represented explicitly in tabular form, and reinforcement learning algorithms can directly estimate these quantities for each state or state-action pair.

In continuous settings, the state space $\mathcal{S} \subseteq \mathbb{R}^n$ and/or the action space $\mathcal{A} \subseteq \mathbb{R}^m$ may be continuous. In such cases, value functions and policies cannot be represented exactly and are instead approximated using parameterized function approximators. A policy may be repre-

sented as $\pi_\theta(a \mid s)$, where θ denotes the parameters of the policy, and a value function may be represented as $V_\phi(s)$ or $Q_\phi(s, a)$, where ϕ denotes the parameters of the function approximator.

Learning in continuous settings involves optimizing the parameters θ and ϕ to maximize expected return. These formulations are particularly important in robotics and control tasks, where states and actions are naturally continuous and policies must operate over high-dimensional spaces [1].

1.3 Hybrid Planning and Learning

Symbolic planning and reinforcement learning provide complementary approaches to sequential decision making. Symbolic planning enables structured reasoning over high-level representations but assumes accurate and complete models of the environment. Reinforcement learning enables agents to learn behaviors through interaction but operates over low-level state representations and often lacks explicit structure. Hybrid planning and learning systems aim to combine these paradigms by integrating symbolic reasoning with learned execution.

In hybrid systems, decision making is performed over two distinct but related representations. The symbolic model is defined by a planning task $\mathcal{T} = \langle \mathcal{F}, \mathcal{O}, \tilde{s}_0, g \rangle$, while the execution model is defined by a Markov decision process $M = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$. The symbolic model operates over discrete, structured representations, whereas the execution model operates over environment states that may be continuous and stochastic.

A key component of hybrid systems is an abstraction function $\phi : \mathcal{S} \to \tilde{\mathcal{S}}$ that maps environment states to symbolic states. This mapping enables symbolic reasoning over high-level representations derived from low-level observations.

Operators in the symbolic model correspond to behaviors executed in the environment. Each operator $o \in \mathcal{O}$ is associated with a policy $\pi_o(a \mid s)$ that defines how the operator is realized in the execution model. Thus, symbolic plans correspond to sequences of policies executed in the environment.

A hybrid planning and learning system can be defined as $\mathcal{H} = \langle \mathcal{T}, M, \phi \rangle$, which integrates symbolic reasoning, environment dynamics, and abstraction between representations.

A central challenge in hybrid systems is the mismatch between the symbolic model and the

execution model. An operator o may be applicable in a symbolic state $\tilde{s}$, i.e., $\tilde{s} \models \psi_o$, but its execution through π_o in a corresponding environment state s may fail to achieve the intended effects. Such discrepancies arise due to incomplete or inaccurate symbolic models, stochastic dynamics, or limitations of learned policies.

This mismatch highlights the need for mechanisms that incorporate feedback from execution to improve decision making, either by refining policies, updating symbolic models, or improving the abstraction between representations. Enabling such interactions is central to the design of hybrid planning and learning systems.

Bibliography

[1] Shivam Goel, Yash Shukla, Vasanth Sarathy, Matthias Scheutz, and Jivko Sinapov. Rapid-learn: A framework for learning to recover for handling novelties in open-world environments. In *IEEE International Conference on Development and Learning (ICDL), London, UK, September 12-15, 2022*, pages 1–8. IEEE, 2022. URL `https://arxiv.org/pdf/2206.12493`.